

Problem: Stock Trading with Multiple Holdings and K Transactions

You are given an array `prices`, where `prices[i]` represents the price of one stock on the `i`-th day.

You are also given an integer `k`.

On each day, you can perform **at most one** of the following operations:

1. Buy one stock.
2. Sell one stock that you are currently holding.
3. Do nothing.

You are allowed to hold **multiple stocks at the same time**.

However, each stock unit must be bought before it can be sold.

A transaction is completed when you sell one stock unit. You can complete at most `k` transactions.

Your task is to return the maximum profit you can achieve.

Important Rules

- You can buy at most one stock on a single day.
- You can sell at most one stock on a single day.
- You can hold multiple stocks at the same time.
- You cannot sell a stock before buying it.
- At most `k` sell operations are allowed.
- You are not required to sell all bought stocks.
- Buying decreases profit by `prices[i]`.
- Selling increases profit by `prices[i]`.

Function Signature

```
int maxProfit(vector<int>& prices, int k);
```

Example 1

```
Input:
prices = [1, 2, 3, 4]
k = 2
```

Output:
4

Explanation

One optimal strategy is:

Day 1: Buy at price 1
Day 2: Buy at price 2
Day 3: Sell at price 3
Day 4: Sell at price 4

Profit:

$$(3 - 1) + (4 - 2) = 4$$

So the answer is 4.

Notice that this is different from the standard stock problem, where only one stock can be held at a time.

Example 2

Input:
prices = [3, 1, 5, 2, 7]
k = 2

Output:
9

Explanation

One optimal strategy is:

Day 2: Buy at price 1
Day 4: Buy at price 2
Day 3: Sell at price 5
Day 5: Sell at price 7

But operations must follow day order, so the valid sequence is:

```
Day 2: Buy at price 1
Day 3: Sell at price 5
Day 4: Buy at price 2
Day 5: Sell at price 7
```

Profit:

$$(5 - 1) + (7 - 2) = 9$$

So the answer is **9**.

Example 3

```
Input:
prices = [1, 5, 2, 6]
k = 2
```

```
Output:
8
```

Explanation

One optimal strategy is:

```
Day 1: Buy at price 1
Day 2: Sell at price 5
Day 3: Buy at price 2
Day 4: Sell at price 6
```

Profit:

$$(5 - 1) + (6 - 2) = 8$$

So the answer is **8**.

Example 4

```
Input:
prices = [1, 2, 3, 4, 5]
k = 3
```

Output:
6

Explanation

Since multiple stocks can be held, we can do:

Day 1: Buy at price 1
Day 2: Buy at price 2
Day 3: Buy at price 3
Day 4: Sell at price 4
Day 5: Sell at price 5

Only 2 transactions are completed because only 2 stocks were sold.

Profit:

$$(4 - 1) + (5 - 2) = 6$$

We cannot sell on another day because there are no more days left.

So the answer is 6.

Example 5

Input:
prices = [5, 4, 3, 2, 1]
k = 2

Output:
0

Explanation

The prices are decreasing every day.

Buying and selling will never give profit.

So the best choice is to do nothing.

Answer is 0.

Constraints

```
1 <= prices.length <= 500
1 <= k <= prices.length
1 <= prices[i] <= 10^4
```

Expected Approach

Use Dynamic Programming.

Since we can hold multiple stocks, the usual stock DP state:

```
dp[index][buy][transactions]
```

is not enough.

We need to track how many stocks are currently held.

A good DP state is:

```
dp[day][transactionsLeft][holding]
```

Where:

```
day = current day
transactionsLeft = how many sell operations are still allowed
holding = how many stocks we currently have
```

At every day, we have three choices:

1. Skip
2. Buy one stock
3. Sell one stock

Recurrence idea:

```
skip = solve(day + 1, transactionsLeft, holding)

buy = -prices[day] + solve(day + 1, transactionsLeft, holding + 1)

sell = prices[day] + solve(day + 1, transactionsLeft - 1, holding - 1)
```

Selling is allowed only when:

```
holding > 0 and transactionsLeft > 0
```

The final answer is:

```
solve(0, k, 0)
```

Time Complexity

```
O(n * k * n)
```

Because:

```
day can be n  
transactionsLeft can be k  
holding can be up to n
```

So the complexity is:

```
O(n^2 * k)
```

Space Complexity

```
O(n * k * n)
```